Milestone 2: Report

Coronavirus Tweet Categorization and Exploration

Group 2: - Gabriel Cruz, Shreeya Kotasthane, Lisa Peterson, Md Main Uddin Rony

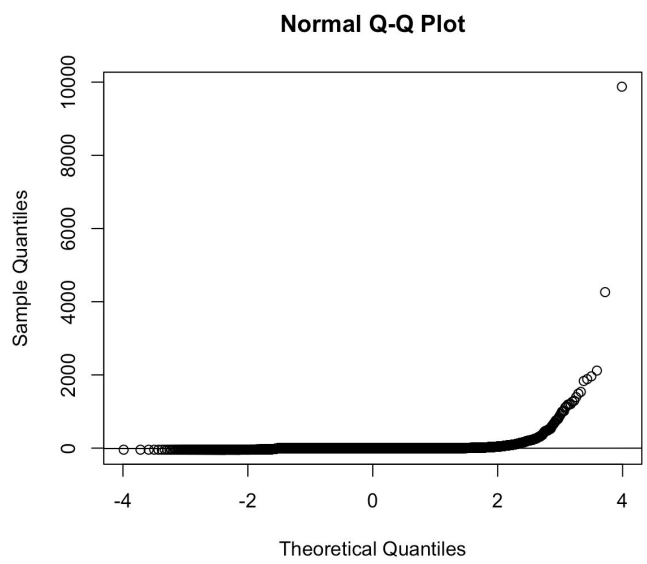
RQ1a - Predicting Tweet Favorites

1. Linear Regressions
    a. Linear Regression Parameters
        i. For user_verified variable:

1.

```
Residuals:
   Min     1Q Median     3Q    Max
 -43.6   -6.1   -6.1   -4.1 9874.9

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)     6.1040     0.9016    6.77 1.33e-11 ***
user_verified  37.4845     3.2335   11.59  < 2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106.2 on 15034 degrees of freedom
Multiple R-squared:  0.00886,   Adjusted R-squared:  0.008794
F-statistic: 134.4 on 1 and 15034 DF,  p-value: < 2.2e-16
```

2.

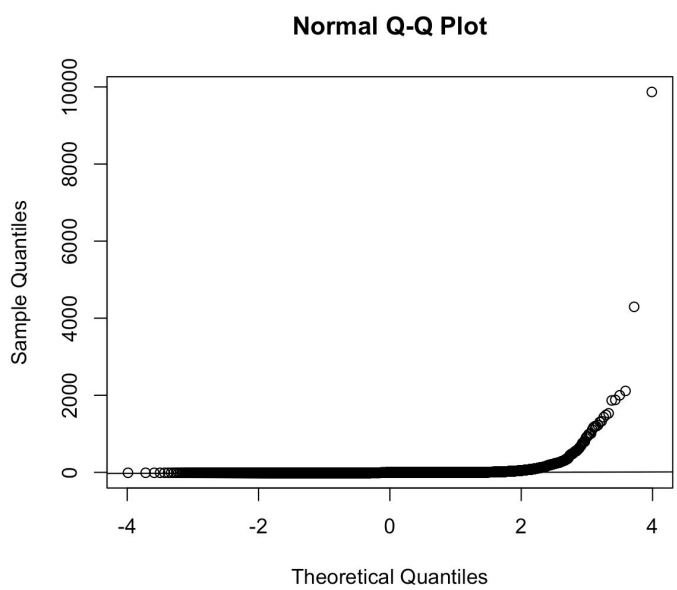


3.
```
> cor(pc,       df_testing$favorite_count)
         [,1]
fit 0.1432059
lwr 0.1432059
upr 0.1432059
> cor(pp,       df_testing$favorite_count)
         [,1]
fit 0.1432059
lwr 0.1432059
upr 0.1432059
```

4. For the user verified variable, while the variable is statistically significant, it is not a very good predictor for likes of a tweet. Note: no confidence band demonstrated because they do not appear in the plots.

ii. For is_quoted_tweet

1.
```
Residuals:
   Min     1Q Median     3Q    Max
 -11.1  -11.1   -6.1   -4.8 9869.9

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)       11.130      1.118   9.956  < 2e-16 ***
is_quoted_tweet   -5.344      1.778  -3.005  0.00266 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106.6 on 15034 degrees of freedom
Multiple R-squared:  0.0006002, Adjusted R-squared:  0.0005337
F-statistic: 9.028 on 1 and 15034 DF,  p-value: 0.002663
```

2.


3.
```
> cor(pc,        df_testing$favorite_count)
          [,1]
fit 0.03633067
lwr 0.03633067
upr 0.03633067
> cor(pp,        df_testing$favorite_count)
          [,1]
fit 0.03633067
lwr 0.03633067
upr 0.03633067
```

4. For the is_quoted_tweet variable, while the variable is statistically significant, it is not a very good predictor for likes of a tweet given the correlation and residuals plot. Note: no confidence band demonstrated because they do not appear in the plots.

iii. For quote_count

1.
```
Residuals:
   Min     1Q Median     3Q    Max
    -9     -9     -8     -7   9872

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   9.0183     0.8697   10.37   <2e-16 ***
quote_count       NA         NA      NA       NA
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106.6 on 15035 degrees of freedom
```

2.
```
> cor(pc,        df_testing$favorite_count)
    [,1]
fit   NA
lwr   NA
upr   NA
Warning message:
In cor(pc, df_testing$favorite_count) : the
> cor(pp,        df_testing$favorite_count)
    [,1]
fit   NA
lwr   NA
upr   NA
```

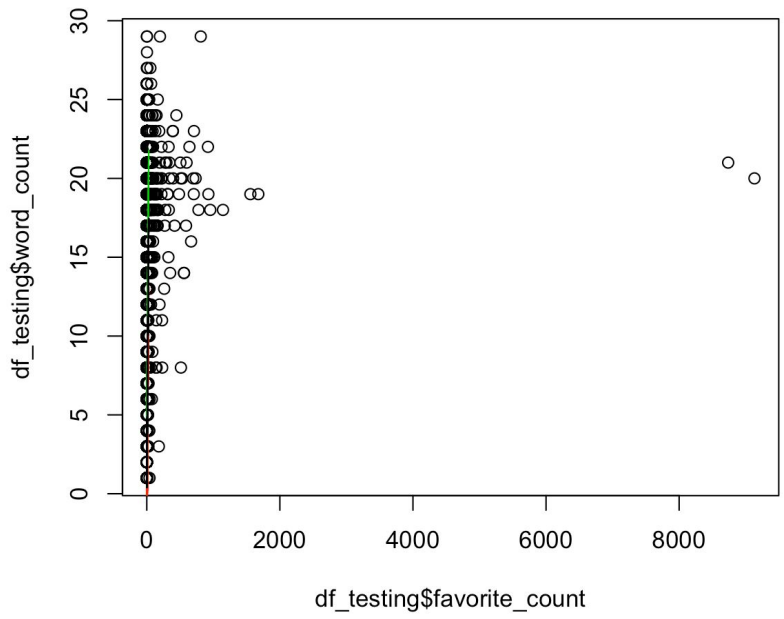
3. Because the data does not lend itself for this kind of application, quote_count does not do a good job of being able to predict anything. In fact, the data does not contain any tweets with quote counts which is why this variable does not do well in predicting anything.

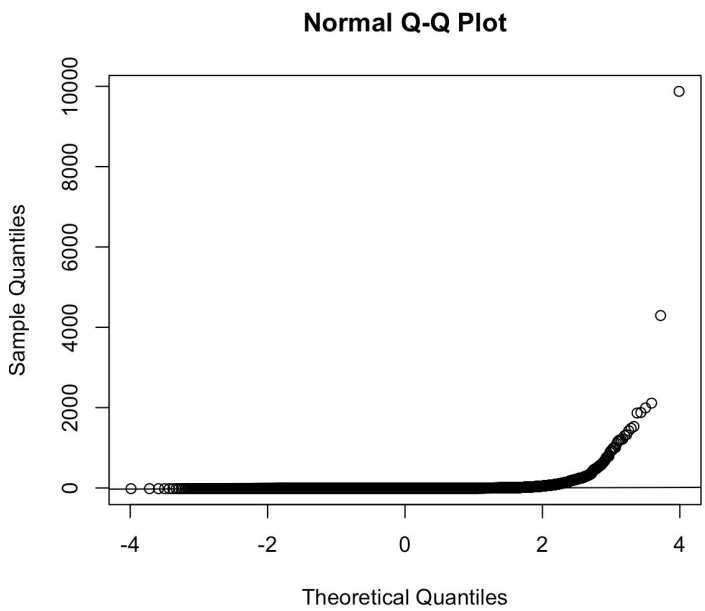
iv. For word_count

1.

```
Residuals:
   Min     1Q Median     3Q    Max 
 -18.1  -10.4   -7.7   -3.5 9873.9 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  -0.1857     2.3813  -0.078    0.938    
word_count    0.6082     0.1465   4.152 3.32e-05 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106.6 on 15034 degrees of freedom
Multiple R-squared:  0.001145,	Adjusted R-squared:  0.001079 
F-statistic: 17.24 on 1 and 15034 DF,  p-value: 3.32e-05
```

2.



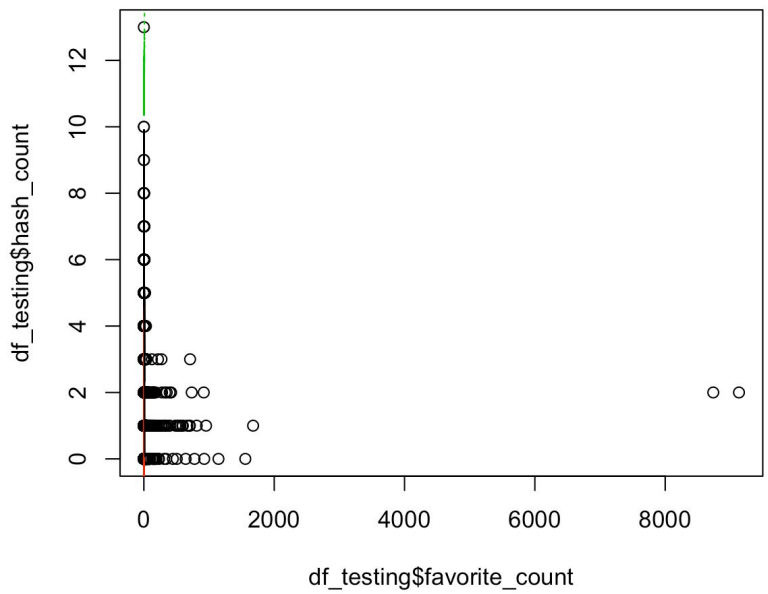
3. 

4.
```
> cor(pc,       df_testing$favorite_count)
          [,1]
fit 0.04272912
lwr 0.04021750
upr 0.04452149
> cor(pp,       df_testing$favorite_count)
          [,1]
fit 0.04272912
lwr 0.04270124
upr 0.04275689
```

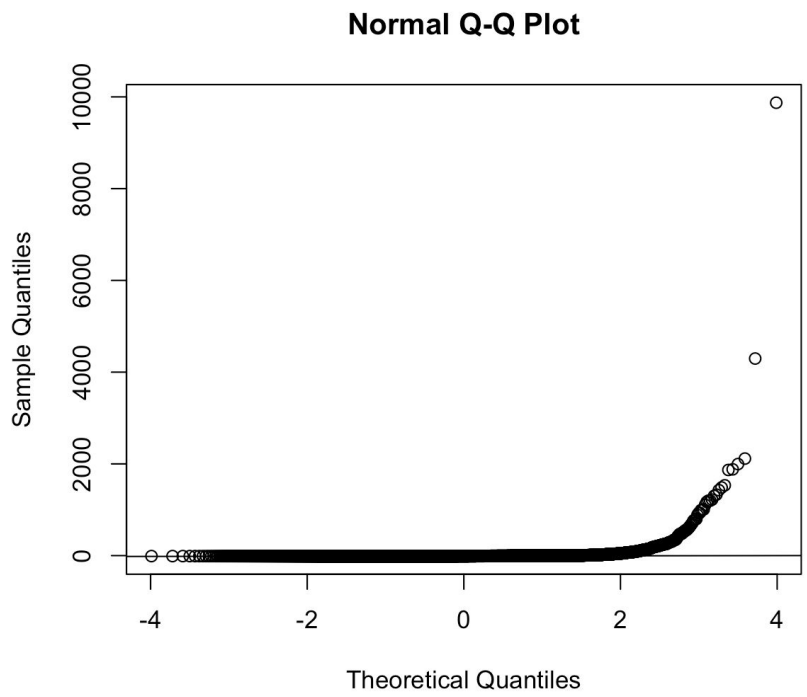
5. For the word_count variable, while the variable is statistically significant, it is not a very good predictor for likes of a tweet given the correlation and residuals plot.

v. For hash_count:

1.
```
Residuals:
   Min     1Q Median     3Q    Max 
  -9.9   -9.1   -8.1   -6.1 9872.6 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)   9.8995     1.1196   8.842   <2e-16 ***
hash_count   -0.7560     0.6049  -1.250    0.211    
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106.6 on 15034 degrees of freedom
Multiple R-squared:  0.0001039, Adjusted R-squared:  3.738e-05 
F-statistic: 1.562 on 1 and 15034 DF,  p-value: 0.2114
```


df_testing$hash_count
0
2
4
6
8
10
12
0
2000
4000
6000
8000
df_testing$favorite_count

2.


Normal Q-Q Plot
Sample Quantiles
0
2000
4000
6000
8000
10000
-4
-2
0
2
4
Theoretical Quantiles

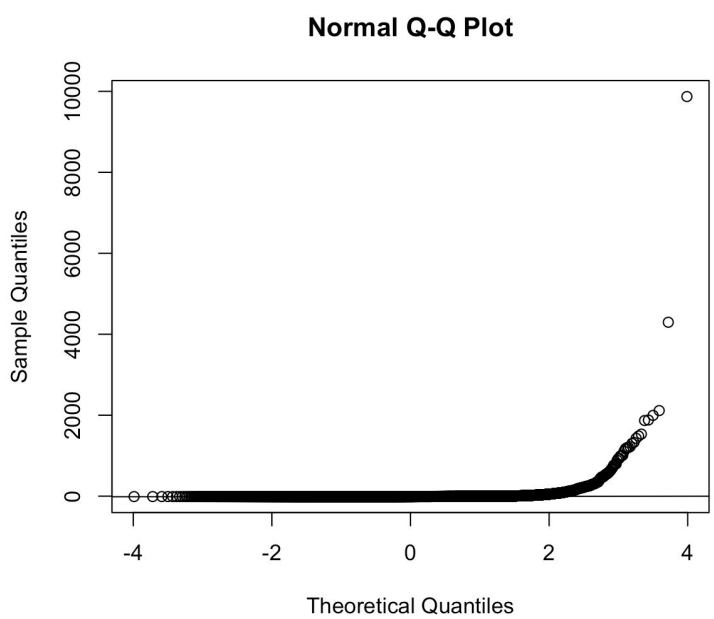
3.

4.
```
> cor(pc,       df_testing$favorite_count)
            [,1]
fit -0.004582012
lwr  0.007192046
upr -0.030729190
> cor(pp,       df_testing$favorite_count)
            [,1]
fit -0.004582012
lwr -0.004279440
upr -0.004890773
```

5. For the hash_count variable, the variable is not a good predictor because the residuals aren't normal, and the regression is not statistically significant.

vi. For sentiment

1.
```
Residuals:
   Min     1Q Median     3Q    Max
 -10.2   -9.0   -8.3   -6.6 9871.7

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   8.9566     0.8762  10.222   <2e-16 ***
sentiment    -1.2187     2.1053  -0.579    0.563
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106.6 on 15034 degrees of freedom
Multiple R-squared:  2.229e-05, Adjusted R-squared:  -4.423e-05
F-statistic: 0.3351 on 1 and 15034 DF,  p-value: 0.5627
```

2.


3.
```
> cor(pc,       df_testing$favorite_count)
             [,1]
fit -0.0054892062
lwr  0.0002923124
upr -0.0069079461
> cor(pp,       df_testing$favorite_count)
            [,1]
fit -0.005489206
lwr -0.005429907
upr -0.005547034
```

4. For the sentiment variable, similar to the hash_count, it is not statistically significant, therefore it does not act as a good predictor for favorite_count.

b. Multivariate Regressions

i. The multivariate regression was created using the three variables that demonstrated statistical significance. They produced the following model. As you can see, the correlations of the model were not very good and demonstrated the deficiencies of the model to predict favorite_count.

1.
```
Call:
lm(formula = favorite_count ~ user_followers_count + user_verified +
    word_count, data = df_training)

Residuals:
   Min     1Q Median     3Q    Max
 -94.3   -7.5   -5.5   -2.2 9876.3

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)          -9.449e-01  2.369e+00  -0.399  0.69005
user_followers_count  6.390e-06  1.091e-06   5.856 4.84e-09 ***
user_verified         2.868e+01  3.511e+00   8.167 3.42e-16 ***
word_count            4.690e-01  1.463e-01   3.205  0.00135 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 106 on 15032 degrees of freedom
Multiple R-squared:  0.01176,   Adjusted R-squared:  0.01157
F-statistic: 59.65 on 3 and 15032 DF,  p-value: < 2.2e-16
```

2.
```
> cor(pc,       df_testing$favorite_count)
         [,1]
fit 0.1416109
lwr 0.1394733
upr 0.1423876
> cor(pp,       df_testing$favorite_count)
         [,1]
fit 0.1416109
lwr 0.1416538
upr 0.1415651
```

ii. Regularization was used in order to improve the multivariate model. Unfortunately the model did not improve the model much and the correlation reported actually changed to NA.

1.
```
> cor(prediction,       as.vector(df_testing[,8]))
  [,1]
1   NA
```

iii.

2. Logistic Regression and NB
   a. Because of the nature of this problem, neither Logistical Regression and NB would be models that we would be able to use for continuous number data. If the data were categorical we might find ourselves using these models, but in our case we are trying to predict a number (the number of likes that a tweet will receive).
3. Decision Trees and Random Forests
   a. The first step in creating a decision tree and random forest model is to categorize the data which at this point is numerical. The next step was to then split the training and testing sets in random order.
      i. In order to prove that the random distribution is similar to the original distribution, a summary was performed on both and the results were similar for both, thus proving the presumption that both need to be similar.

1.
```
                                  :20028
user_verified     user_followers_count is_quoted_tweet   quote_count  reply_count
Min.   :0.00000   Min.   :       0     Min.   :0.0000   Min.   :0    Min.   :0
1st Qu.:0.00000   1st Qu.:      96     1st Qu.:0.0000   1st Qu.:0    1st Qu.:0
Median :0.00000   Median :     601     Median :0.0000   Median :0    Median :0
Mean   :0.07776   Mean   :   96964     Mean   :0.3897   Mean   :0    Mean   :0
3rd Qu.:0.00000   3rd Qu.:    2682     3rd Qu.:1.0000   3rd Qu.:0    3rd Qu.:0
Max.   :1.00000   Max.   :14068695     Max.   :1.0000   Max.   :0    Max.   :0

retweet_count       favorite_count       word_count      hash_count       sentiment
Min.   :    0.000   Min.   :   0.000   Min.   : 1.00   Min.   : 0.000   Min.   :-0.9846
1st Qu.:    0.000   1st Qu.:   0.000   1st Qu.:11.00   1st Qu.: 0.000   1st Qu.:-0.3818
Median :    0.000   Median :   1.000   Median :17.00   Median : 1.000   Median : 0.0000
Mean   :    5.432   Mean   :   9.943   Mean   :15.16   Mean   : 1.161   Mean   :-0.0500
3rd Qu.:    1.000   3rd Qu.:   2.000   3rd Qu.:20.00   3rd Qu.: 2.000   3rd Qu.: 0.2263
Max.   :14511.000   Max.   :9881.000   Max.   :30.00   Max.   :14.000   Max.   : 0.9740
```

2.

```
                                  :20028
user_verified     user_followers_count is_quoted_tweet   quote_count  reply_count
Min.   :0.00000   Min.   :        0     Min.   :0.0000   Min.   :0    Min.   :0
1st Qu.:0.00000   1st Qu.:       96     1st Qu.:0.0000   1st Qu.:0    1st Qu.:0
Median :0.00000   Median :      601     Median :0.0000   Median :0    Median :0
Mean   :0.07776   Mean   :    96964     Mean   :0.3897   Mean   :0    Mean   :0
3rd Qu.:0.00000   3rd Qu.:     2682     3rd Qu.:1.0000   3rd Qu.:0    3rd Qu.:0
Max.   :1.00000   Max.   :14068695      Max.   :1.0000   Max.   :0    Max.   :0

retweet_count       favorite_count       word_count      hash_count       sentiment
Min.   :    0.000   Min.   :   0.000   Min.   : 1.00   Min.   : 0.000   Min.   :-0.9846
1st Qu.:    0.000   1st Qu.:   0.000   1st Qu.:11.00   1st Qu.: 0.000   1st Qu.:-0.3818
Median :    0.000   Median :   1.000   Median :17.00   Median : 1.000   Median : 0.0000
Mean   :    5.432   Mean   :   9.943   Mean   :15.16   Mean   : 1.161   Mean   :-0.0500
3rd Qu.:    1.000   3rd Qu.:   2.000   3rd Qu.:20.00   3rd Qu.: 2.000   3rd Qu.: 0.2263
Max.   :14511.000   Max.   :9881.000   Max.   :30.00   Max.   :14.000   Max.   : 0.9740
```

ii. The decision tree demonstrates a clear bias for assuming that the likes that a tweet will receive will be low almost every single time. This is demonstrated by the decision matrix where the only class predicted is A. The errors in the decision tree are 8.1 and the size is 1.

1.

```
   (a)   (b)   (c)   (d)   (e)    <-classified as
  ----  ----  ----  ----  ----
 13818                             (a): class likes1
  1004                             (b): class likes2
   195                             (c): class likes3
    19                             (d): class likes4
                                   (e): class likes5
```

iii. Unfortunately the prediction only predicted for the one class and therefor the error rate will be the amount of cases that are not class likes 1. The CrossTable was created and demonstrated that only 91% of instances were correctly predicted because 91% of the cases were class likes1.

1.

```
                                | tree_predict
df_testing_random$favorite_count |    likes1 | Row Total |
---------------------------------|-----------|-----------|
                         likes1 |      4574 |      4574 |
                                |     0.913 |           |
---------------------------------|-----------|-----------|
                         likes2 |       358 |       358 |
                                |     0.071 |           |
---------------------------------|-----------|-----------|
                         likes3 |        75 |        75 |
                                |     0.015 |           |
---------------------------------|-----------|-----------|
                         likes4 |         5 |         5 |
                                |     0.001 |           |
---------------------------------|-----------|-----------|
                   Column Total |      5012 |      5012 |
---------------------------------|-----------|-----------|
```

iv. Boosting was done in hopes of some sort of change for better prediction models. Unfortunately, boosting did not change the model at all and resulted in the exact same results.

v. However, bagging performed leaps and bounds ahead of what was expected. In the case of bagging, the model actually predicted other things that weren't likes1 and did so with error but not so much so that I couldnt confidently state that this model is better than the previous models.

1.

```
Total Observations in Table:  5012


                                 | pred
df_testing_random$favorite_count |    likes1 |    likes2 |    likes3 |    likes4 | Row Total |
---------------------------------|-----------|-----------|-----------|-----------|-----------|
                          likes1 |      4526 |        48 |         0 |         0 |      4574 |
                                 |     9.794 |   120.558 |    43.805 |     4.563 |           |
                                 |     0.990 |     0.010 |     0.000 |     0.000 |     0.913 |
                                 |     0.956 |     0.213 |     0.000 |     0.000 |           |
                                 |     0.903 |     0.010 |     0.000 |     0.000 |           |
---------------------------------|-----------|-----------|-----------|-----------|-----------|
                          likes2 |       206 |       139 |        13 |         0 |       358 |
                                 |    51.640 |   940.267 |    26.720 |     0.357 |           |
                                 |     0.575 |     0.388 |     0.036 |     0.000 |     0.071 |
                                 |     0.044 |     0.618 |     0.271 |     0.000 |           |
                                 |     0.041 |     0.028 |     0.003 |     0.000 |           |
---------------------------------|-----------|-----------|-----------|-----------|-----------|
                          likes3 |         2 |        38 |        33 |         2 |        75 |
                                 |    66.896 |   356.246 |  1450.848 |    49.536 |           |
                                 |     0.027 |     0.507 |     0.440 |     0.027 |     0.015 |
                                 |     0.000 |     0.169 |     0.688 |     0.400 |           |
                                 |     0.000 |     0.008 |     0.007 |     0.000 |           |
---------------------------------|-----------|-----------|-----------|-----------|-----------|
                          likes4 |         0 |         0 |         2 |         3 |         5 |
                                 |     4.723 |     0.224 |    79.581 |  1798.325 |           |
                                 |     0.000 |     0.000 |     0.400 |     0.600 |     0.001 |
                                 |     0.000 |     0.000 |     0.042 |     0.600 |           |
                                 |     0.000 |     0.000 |     0.000 |     0.001 |           |
---------------------------------|-----------|-----------|-----------|-----------|-----------|
                    Column Total |      4734 |       225 |        48 |         5 |      5012 |
                                 |     0.945 |     0.045 |     0.010 |     0.001 |           |
---------------------------------|-----------|-----------|-----------|-----------|-----------|
```

2. With 10 trees:

a.

	pred				
df_testing_random$favorite_count	likes1	likes2	likes3	likes4	Row Total
likes1	4515	59	0	0	4574
	9.549	106.805	49.281	4.563	
	0.987	0.013	0.000	0.000	0.913
	0.956	0.259	0.000	0.000	
	0.901	0.012	0.000	0.000	
likes2	208	132	18	0	358
	49.690	822.180	51.857	0.357	
	0.581	0.369	0.050	0.000	0.071
	0.044	0.579	0.333	0.000	
	0.042	0.026	0.004	0.000	
likes3	2	37	34	2	75
	66.762	330.665	1363.394	49.536	
	0.027	0.493	0.453	0.027	0.015
	0.000	0.162	0.630	0.400	
	0.000	0.007	0.007	0.000	
likes4	0	0	2	3	5
	4.714	0.227	70.306	1798.325	
	0.000	0.000	0.400	0.600	0.001
	0.000	0.000	0.037	0.600	
	0.000	0.000	0.000	0.001	
Column Total	4725	228	54	5	5012
	0.943	0.045	0.011	0.001	

3. With 20 trees:

a.

	pred				
df_testing_random$favorite_count	likes1	likes2	likes3	likes4	Row Total
likes1	4518	56	0	0	4574
	10.285	117.936	46.543	4.563	
	0.988	0.012	0.000	0.000	0.913
	0.957	0.237	0.000	0.000	
	0.901	0.011	0.000	0.000	
likes2	200	143	15	0	358
	55.787	943.933	35.408	0.357	
	0.559	0.399	0.042	0.000	0.071
	0.042	0.606	0.294	0.000	
	0.040	0.029	0.003	0.000	
likes3	2	37	34	2	75
	66.687	317.183	1447.501	49.536	
	0.027	0.493	0.453	0.027	0.015
	0.000	0.157	0.667	0.400	
	0.000	0.007	0.007	0.000	
likes4	0	0	2	3	5
	4.709	0.235	74.670	1798.325	
	0.000	0.000	0.400	0.600	0.001
	0.000	0.000	0.039	0.600	
	0.000	0.000	0.000	0.001	
Column Total	4720	236	51	5	5012
	0.942	0.047	0.010	0.001	

4. With 30 trees:

a.

	pred				
df_testing_random$favorite_count	likes1	likes2	likes3	likes4	Row Total
likes1	4520	54	0	0	4574
	9.846	114.941	45.630	5.476	
	0.988	0.012	0.000	0.000	0.913
	0.956	0.236	0.000	0.000	
	0.902	0.011	0.000	0.000	
likes2	205	136	17	0	358
	52.109	875.117	50.491	0.429	
	0.573	0.380	0.047	0.000	0.071
	0.043	0.594	0.340	0.000	
	0.041	0.027	0.003	0.000	
likes3	2	39	31	3	75
	66.792	369.284	1223.157	94.330	
	0.027	0.520	0.413	0.040	0.015
	0.000	0.170	0.620	0.500	
	0.000	0.008	0.006	0.001	
likes4	0	0	2	3	5
	4.716	0.228	76.242	1497.606	
	0.000	0.000	0.400	0.600	0.001
	0.000	0.000	0.040	0.500	
	0.000	0.000	0.000	0.001	
Column Total	4727	229	50	6	5012
	0.943	0.046	0.010	0.001	

5. With 100 trees:

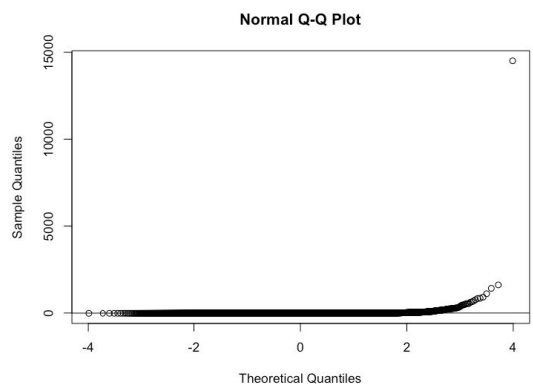
a.

	pred				
df_testing_random$favorite_count	likes1	likes2	likes3	likes4	Row Total
likes1	4524	50	0	0	4574
	9.782	115.796	47.456	4.563	
	0.989	0.011	0.000	0.000	0.913
	0.956	0.224	0.000	0.000	
	0.903	0.010	0.000	0.000	
likes2	206	135	17	0	358
	51.550	890.099	47.522	0.357	
	0.575	0.377	0.047	0.000	0.071
	0.044	0.605	0.327	0.000	
	0.041	0.027	0.003	0.000	
likes3	2	38	33	2	75
	66.867	360.062	1334.283	49.536	
	0.027	0.507	0.440	0.027	0.015
	0.000	0.170	0.635	0.400	
	0.000	0.008	0.007	0.000	
likes4	0	0	2	3	5
	4.721	0.222	73.160	1798.325	
	0.000	0.000	0.400	0.600	0.001
	0.000	0.000	0.038	0.600	
	0.000	0.000	0.000	0.001	
Column Total	4732	223	52	5	5012
	0.944	0.044	0.010	0.001	

4. Out of all of the models for predicting the likes, the model that performed the best was the random forest. Unfortunately, linear regression demonstrated very poor performance. While most of the relevant variables were statistically significant, there really was no clear prediction effect from the variables. Meaning, while they were statistically significant, they really did not do a good job predicting. This is demonstrated clearly by the multivariate regression, there the model does not do a good job. Decision trees did not do a good job predicting and did not even predict anything, the models appeared to

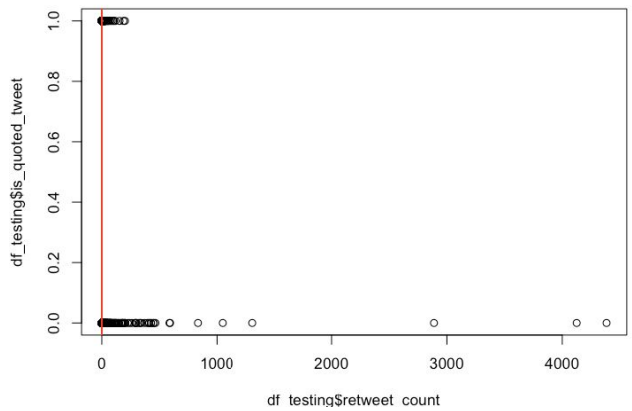
only be guessing. In the case of random forests with bagging, the model did a good job actually choosing which category of likes a tweet will fall into. However, even if it was the best model, there seemed to be some guessing going on wherever the category was not likes1, it seemed to be that anything other than likes 1 was a tossup.

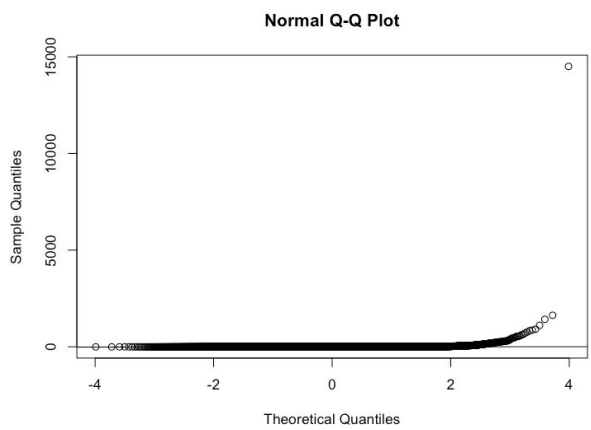
RQ1b - Predicting Tweet Retweets

1. Linear Regressions
    a. Linear Regression Parameters
        i. For user_verfiied variable
        1.
```
Residuals:
    Min      1Q  Median      3Q     Max
  -18.0    -3.8    -3.8    -2.8 14507.2

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)      3.791      1.043   3.637 0.000277 ***
user_verified   14.207      3.739   3.800 0.000145 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 122.8 on 15034 degrees of freedom
Multiple R-squared:  0.0009594, Adjusted R-squared:  0.000893
F-statistic: 14.44 on 1 and 15034 DF,  p-value: 0.0001454
```
        2.


        3.
```
> cor(pc, df_testing$retweet_count)
          [,1]
fit 0.1444409
lwr 0.1444409
upr 0.1444409
> cor(pp, df_testing$retweet_count)
          [,1]
fit 0.1444409
lwr 0.1444409
upr 0.1444409
```
        4. For the user verified variable, while the variable is statistically significant, it is not a very good predictor for likes of a tweet. Note: no confidence band demonstrated because they do not appear in the plots.
        ii. For is_quoted_tweet variable

1.
```
Residuals:
    Min      1Q  Median      3Q     Max
   -6.4    -6.4    -4.4    -2.6 14504.6

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)        6.408      1.288   4.976 6.57e-07 ***
is_quoted_tweet   -3.826      2.049  -1.868   0.0618 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 122.8 on 15034 degrees of freedom
Multiple R-squared:  0.000232,  Adjusted R-squared:  0.0001655
F-statistic: 3.488 on 1 and 15034 DF,  p-value: 0.06182
```

2.


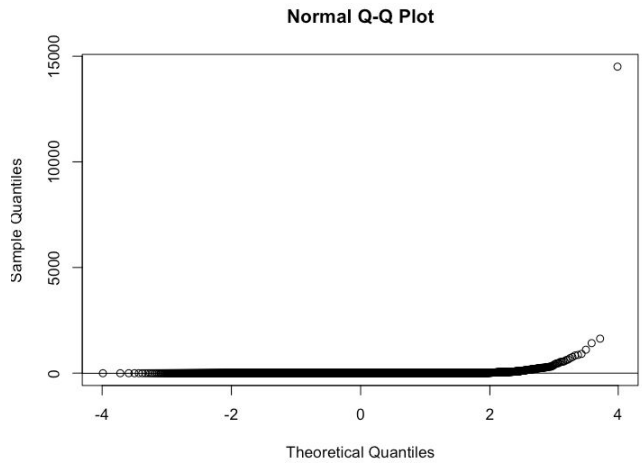
3.


4.
```
> cor(pc, df_testing$retweet_count)
          [,1]
fit 0.04100787
lwr 0.04100787
upr 0.04100787
> cor(pp, df_testing$retweet_count)
          [,1]
fit 0.04100787
lwr 0.04100787
upr 0.04100787
```

5. The user verified is not a predictor of retweet count. In fact it is not even statistically significant and has a very low correlation.

iii. For quote_count variable

1.
```
Residuals:
    Min      1Q  Median      3Q     Max
   -4.9    -4.9    -4.9    -3.9 14506.1

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    4.896      1.002   4.888 1.03e-06 ***
quote_count       NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 122.8 on 15035 degrees of freedom
```

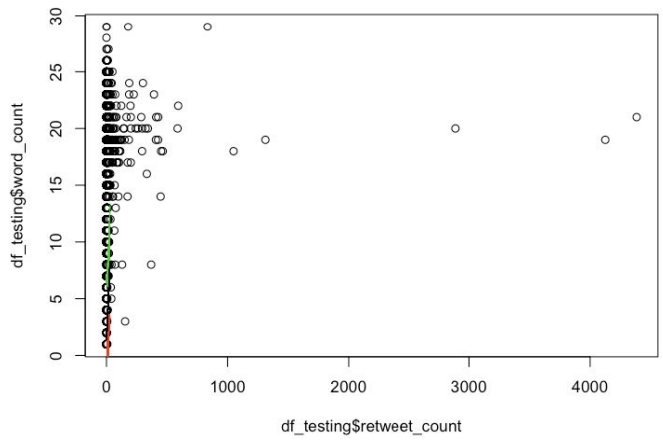
2.


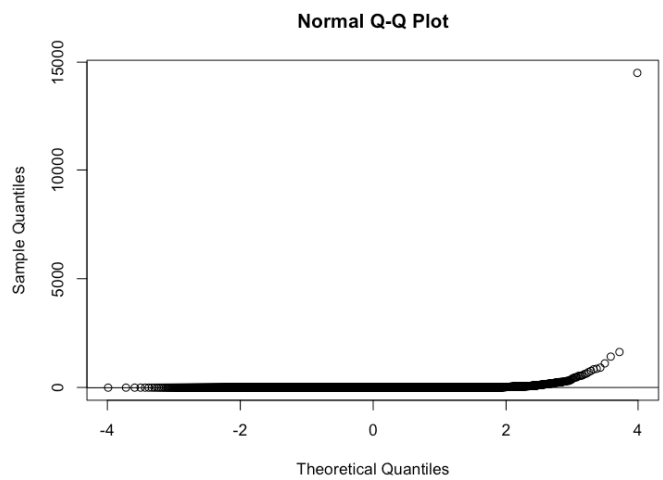
3.
```
> cor(pc, df_testing$retweet_count)
    [,1]
fit   NA
lwr   NA
upr   NA
Warning message:
In cor(pc, df_testing$retweet_count) : the standard deviation is zero
> cor(pp, df_testing$retweet_count)
    [,1]
fit   NA
lwr   NA
upr   NA
Warning message:
In cor(pp, df_testing$retweet_count) : the standard deviation is zero
```

4. Quote count is not a predictor of retweets and does not even produce a model.

iv. For word_count

1.
```
Residuals:
    Min      1Q  Median      3Q     Max
   -8.5    -5.6    -4.4    -2.7 14506.9

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.2191     2.7439   0.444    0.657
word_count    0.2430     0.1688   1.439    0.150

Residual standard error: 122.8 on 15034 degrees of freedom
Multiple R-squared:  0.0001378, Adjusted R-squared:  7.127e-05
F-statistic: 2.072 on 1 and 15034 DF,  p-value: 0.1501
```

2. 

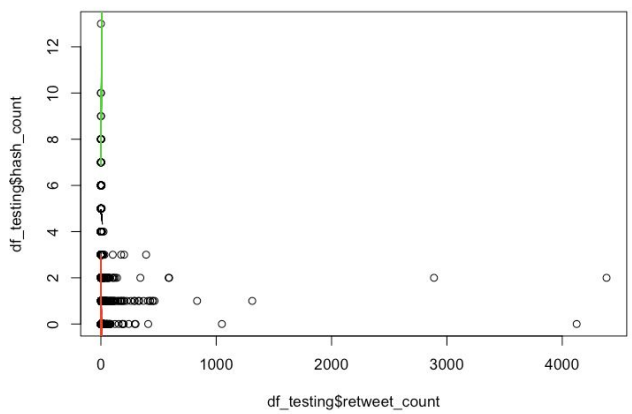
3. 

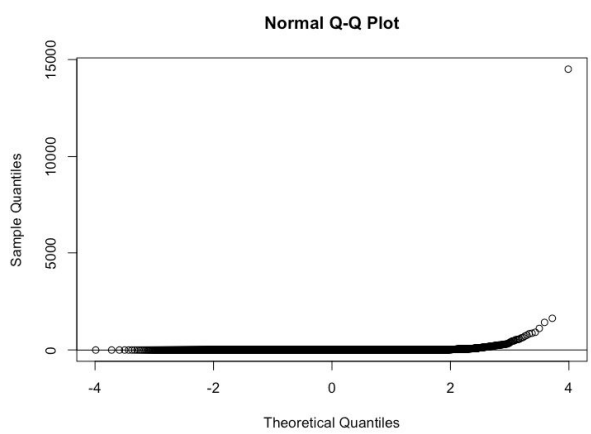
4.
```
> cor(pc, df_testing$retweet_count)
          [,1]
fit 0.04608071
lwr 0.03743750
upr 0.04817346
> cor(pp, df_testing$retweet_count)
          [,1]
fit 0.04608071
lwr 0.04598087
upr 0.04617962
```

5. Word count as demonstrated does not work as a predictor of retweet count. It also does not provide a good correlation.

v. For hashtag count

1.
```
Residuals:
    Min      1Q  Median      3Q     Max
   -5.0    -5.0    -4.9    -3.9 14506.1

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  4.95183    1.28950   3.840 0.000123 ***
hash_count  -0.04797    0.69672  -0.069 0.945108
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 122.8 on 15034 degrees of freedom
Multiple R-squared:  3.153e-07, Adjusted R-squared:  -6.62e-05
F-statistic: 0.004741 on 1 and 15034 DF,  p-value: 0.9451
```

2.


3.


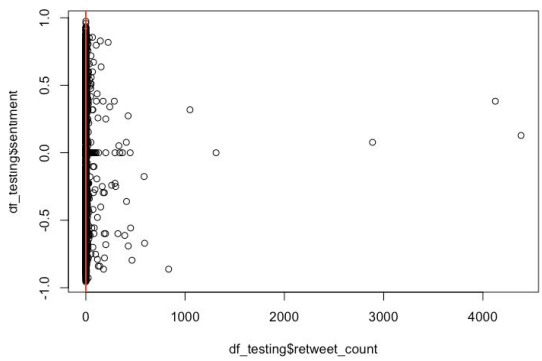
4.
```
> cor(pc, df_testing$retweet_count)
             [,1]
fit  0.001376071
lwr  0.019759696
upr -0.021697947
> cor(pp, df_testing$retweet_count)
             [,1]
fit  0.001376071
lwr  0.004895731
upr -0.003899689
```

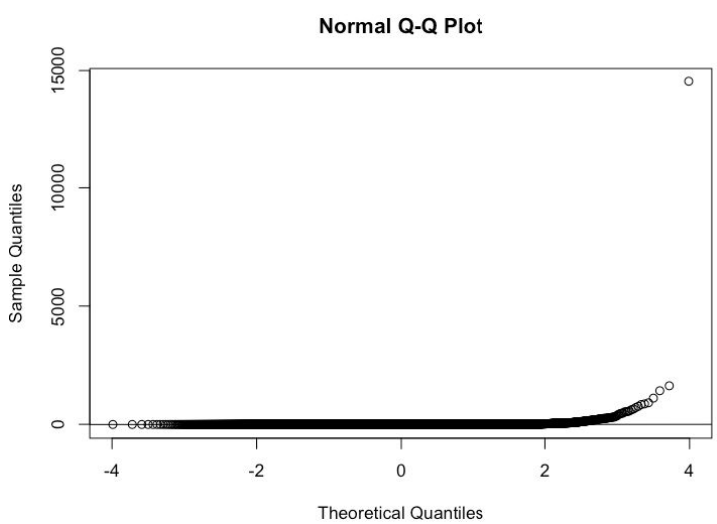
5. Hashtag count does not do a good job of being a predictor of retweets, however, it does demonstrate some correlation. Again

the residuals do not demonstrate normal distribution and therefore I would not be comfortable predicting retweets based on hashtag count.

vi. For sentiment variabel

1.
```
Residuals:
    Min      1Q  Median      3Q     Max
   -6.8    -5.0    -4.6    -3.4 14505.6

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    4.795      1.009   4.752 2.04e-06 ***
sentiment     -1.993      2.425  -0.822    0.411
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 122.8 on 15034 degrees of freedom
Multiple R-squared:  4.494e-05, Adjusted R-squared:  -2.158e-05
F-statistic: 0.6756 on 1 and 15034 DF,  p-value: 0.4111
```

2.


3.


4.
```
> cor(pc, df_testing$retweet_count)
            [,1]
fit -0.003936407
lwr -0.000893229
upr -0.004869688
> cor(pp, df_testing$retweet_count)
            [,1]
fit -0.003936407
lwr -0.003898848
upr -0.003973441
```
5. Sentiment does not do a good job of being a predictor of retweets, however, it does demonstrate some correlation. Again the residuals do not demonstrate normal distribution and therefore I would not be comfortable predicting retweets based on hashtag count.

b. Multivariate Regressions

i. The multivariate regression was created using the same three variables that demonstrated statistical significance in the tweet likes model. They produced the following model. As you can see, the correlations of the model were not very good and demonstrated the deficiencies of the model to predict retweet_count.

1.
```
Residuals:
    Min      1Q  Median      3Q     Max 
  -38.2    -4.5    -3.5    -2.1 14507.8 

Coefficients:
                     Estimate Std. Error t value Pr(>|t|)    
(Intercept)          9.389e-01  2.743e+00   0.342  0.73220    
user_followers_count 2.132e-06  1.263e-06   1.688  0.09145 .  
user_verified        1.120e+01  4.065e+00   2.755  0.00587 ** 
word_count           1.899e-01  1.694e-01   1.121  0.26232    
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 122.8 on 15032 degrees of freedom
Multiple R-squared:  0.00123,	Adjusted R-squared:  0.00103 
F-statistic: 6.169 on 3 and 15032 DF,  p-value: 0.0003475
```

ii. Correlations also demonstrate that there is no real predicting ability from the multivariate model to try to make a prediction on the amount of retweets that a tweet will get.

1.
```
> cor(pc, df_testing$retweet_count)
         [,1]
fit 0.1394196
lwr 0.1230881
upr 0.1389572
> cor(pp, df_testing$retweet_count)
         [,1]
fit 0.1394196
lwr 0.1397030
upr 0.1391113
```
.

iii. Regularization was applied in order to determine if it would help create a model. Unfortunately, regularization did not aid in any way and instead

failed to create any preciuting model at all. The correlation was determined to be as 0.

2. Logistic Regression and NB
    a. Because of the nature of this problem, neither Logistical Regression and NB would be models that we would be able to use for continuous number data. If the data were categorical we might find ourselves using these models, but in our case we are trying to predict a number (the number of likes that a tweet will receive).
3. Decision Trees and Random Forests
    a. The first step in creating a decision tree and random forest model is to categorize the data which at this point is numerical. The next step was to then split the training and testing sets in random order.
    b. Much like the likes question, byt virtue of answering similar question the dataset was the exact same, so this means that the only thing we had to change in the models was to make sure that instead of tweet likes we are predicting tweet retweets.
    c. Much like predicting likes, the models did a bad job of predicting. In the case of the tree, the tree itself did a bad job trying to predict which item to classify. In this case, it decided to only classifty on one category meaning that it ignored any other categories completely and essentially was only guessing.
        i.

```
Evaluation on training data (15036 cases):

          Decision Tree
        ----------------
        Size      Errors

           1  711( 4.7%)   <<


         (a)   (b)   (c)   (d)   (e)    <-classified as
        ----  ----  ----  ----  ----
       14325                             (a): class likes1
         593                             (b): class likes2
         114                             (c): class likes3
           4                             (d): class likes4
                                         (e): class likes5
```

    d. The cross table demonstrates this more clearly:
        i.

df_testing_random$retweet_count	tree_predict retweets1	Row Total
retweets1	4754 0.949	4754
retweets2	219 0.044	219
retweets3	37 0.007	37
retweets4	2 0.000	2
Column Total	5012	5012

    e. Unfortunately boosting did not do anything to aid the situation and instead only repeated the problem again. Below is the CrossTable for reference.

i.

df_testing_random$retweet_count	tree_predict_10 retweets1	Row Total
retweets1	4754 0.949	4754
retweets2	219 0.044	219
retweets3	37 0.007	37
retweets4	2 0.000	2
Column Total	5012	5012

f. However, bagging performed pretty well on the data and even produced favorable results. The crosstable for those results are demonstrated below:

i.

df_testing_random$retweet_count	pred retweets1	retweets2	retweets3	retweets4	Row Total
retweets1	4709 6.175 0.991 0.984 0.940	45 101.456 0.009 0.237 0.009	0 31.301 0.000 0.000 0.000	0 0.949 0.000 0.000 0.000	4754 0.949
retweets2	79 81.043 0.361 0.016 0.016	130 1783.938 0.594 0.684 0.026	10 50.793 0.046 0.303 0.002	0 0.044 0.000 0.000 0.000	219 0.044
retweets3	0 35.346 0.000 0.000 0.000	15 131.815 0.405 0.079 0.003	22 1942.982 0.595 0.667 0.004	0 0.007 0.000 0.000 0.000	37 0.007
retweets4	0 1.911 0.000 0.000 0.000	0 0.076 0.000 0.000 0.000	1 73.953 0.500 0.030 0.000	1 2504.000 0.500 1.000 0.000	2 0.000
Column Total	4788 0.955	190 0.038	33 0.007	1 0.000	5012 0.955

ii. With 10 trees:

1.

df_testing_random$retweet_count	pred retweets1	retweets2	retweets3	retweets4	Row Total
retweets1	4708	46	0	0	4754
	6.389	101.735	33.198	0.949	
	0.990	0.010	0.000	0.000	0.949
	0.984	0.240	0.000	0.000	
	0.939	0.009	0.000	0.000	
retweets2	76	133	10	0	219
	84.669	1850.867	46.917	0.044	
	0.347	0.607	0.046	0.000	0.044
	0.016	0.693	0.286	0.000	
	0.015	0.027	0.002	0.000	
retweets3	0	13	24	0	37
	35.317	94.650	2181.534	0.007	
	0.000	0.351	0.649	0.000	0.007
	0.000	0.068	0.686	0.000	
	0.000	0.003	0.005	0.000	
retweets4	0	0	1	1	2
	1.909	0.077	69.614	2504.000	
	0.000	0.000	0.500	0.500	0.000
	0.000	0.000	0.029	1.000	
	0.000	0.000	0.000	0.000	
Column Total	4784	192	35	1	5012
	0.955	0.038	0.007	0.000	

iii. With 20 trees:

1.

df_testing_random$retweet_count	pred retweets1	retweets2	retweets3	retweets4	Row Total
retweets1	4713	41	0	0	4754
	6.474	107.547	31.301	0.949	
	0.991	0.009	0.000	0.000	0.949
	0.984	0.216	0.000	0.000	
	0.940	0.008	0.000	0.000	
retweets2	75	135	9	0	219
	86.099	1933.536	39.616	0.044	
	0.342	0.616	0.041	0.000	0.044
	0.016	0.711	0.273	0.000	
	0.015	0.027	0.002	0.000	
retweets3	0	14	23	0	37
	35.346	113.140	2125.700	0.007	
	0.000	0.378	0.622	0.000	0.007
	0.000	0.074	0.697	0.000	
	0.000	0.003	0.005	0.000	
retweets4	0	0	1	1	2
	1.911	0.076	73.953	2504.000	
	0.000	0.000	0.500	0.500	0.000
	0.000	0.000	0.030	1.000	
	0.000	0.000	0.000	0.000	
Column Total	4788	190	33	1	5012
	0.955	0.038	0.007	0.000	

iv. With 30 trees:

1.

df_testing_random$retweet_count	pred retweets1	retweets2	retweets3	retweets4	Row Total
retweets1	4709	45	0	0	4754
	5.894	96.131	33.198	0.949	
	0.991	0.009	0.000	0.000	0.949
	0.983	0.245	0.000	0.000	
	0.940	0.009	0.000	0.000	
retweets2	83	126	10	0	219
	76.288	1730.690	46.917	0.044	
	0.379	0.575	0.046	0.000	0.044
	0.017	0.685	0.286	0.000	
	0.017	0.025	0.002	0.000	
retweets3	0	13	24	0	37
	35.376	99.775	2181.534	0.007	
	0.000	0.351	0.649	0.000	0.007
	0.000	0.071	0.686	0.000	
	0.000	0.003	0.005	0.000	
retweets4	0	0	1	1	2
	1.912	0.073	69.614	2504.000	
	0.000	0.000	0.500	0.500	0.000
	0.000	0.000	0.029	1.000	
	0.000	0.000	0.000	0.000	
Column Total	4792	184	35	1	5012
	0.956	0.037	0.007	0.000	

v. With 100 trees:

1.

df_testing_random$retweet_count	pred retweets1	retweets2	retweets3	retweets4	Row Total
retweets1	4713	41	0	0	4754
	6.044	103.056	30.353	0.949	
	0.991	0.009	0.000	0.000	0.949
	0.983	0.222	0.000	0.000	
	0.940	0.008	0.000	0.000	
retweets2	81	130	8	0	219
	78.796	1838.736	31.170	0.044	
	0.370	0.594	0.037	0.000	0.044
	0.017	0.703	0.250	0.000	
	0.016	0.026	0.002	0.000	
retweets3	0	14	23	0	37
	35.391	116.880	2193.550	0.007	
	0.000	0.378	0.622	0.000	0.007
	0.000	0.076	0.719	0.000	
	0.000	0.003	0.005	0.000	
retweets4	0	0	1	1	2
	1.913	0.074	76.325	2504.000	
	0.000	0.000	0.500	0.500	0.000
	0.000	0.000	0.031	1.000	
	0.000	0.000	0.000	0.000	
Column Total	4794	185	32	1	5012
	0.957	0.037	0.006	0.000	

vi. As you can see, model performance did not obviously improve with the amount of trees, however it appears that the precision did increase and it was able to predict the category of the retweets with good accuracy.

4. Comparative Analysis
   a. Much like the tweet likes, out of all of the models for predicting the retweets, the model that performed the best was the random forest with bagging. Unfortunately, linear regression demonstrated very poor performance. While most of the relevant variables were statistically significant, there really was no clear prediction effect from the variables. Meaning, while they were statistically significant, they really did not do a good job predicting. This is demonstrated clearly by the multivariate regression, there the model does not do a good job predicting at all. Decision trees did not do a good job predicting and did not even predict anything, the models appeared to only be guessing. In the case of

random forests with bagging, the model did a good job actually choosing which category of likes a tweet will fall into.

2- LOGISTIC REGRESSION AND NB

4 - COMPARATIVE ANALYSIS

RQ2: Classifying Conspiracy Tweets

Research question 2 asks if we can predict if a tweet is a conspiracy type based on the tweet text. During the COVID-19 epidemic, a lot of conspiracy theories surfaced over social media. Conspiracy theories often result in fake news, so automatic identification of them can help to prevent them from spreading.

One of the group members manually annotated 835 tweets into two categories, Conspiracy, and Non-Conspiracy. Tweets that contain information, for example, "Coronavirus spread from China's lab", "China made this virus as a bioweapon" or "China is hiding real affected numbers", etc. are considered as conspiracy tweets. The distribution of annotated tweets is as follow:

Conspiracy Tweets	Non-Conspiracy Tweets
88	747

Table 1: The original distribution of the conspiracy tweets

As we can see, the dataset is an imbalanced one, machine learning algorithms trained on this dataset may produce unsatisfactory results. The algorithms may ignore minority classes to increase the overall performance. To tackle this problem, we have used the over-sampling method, where each conspiracy tweet has been repeated 5 times to make the dataset more balanced. Then the whole dataset has been shuffled to avoid the problem that may arise from order. After over-sampling, the distribution changes to:

Conspiracy Tweets	Non-Conspiracy Tweets
440	747
	Total: 1187

Table 2: The distribution of the conspiracy tweets after over sampling

Tweet Preprocessing

We used a python package, tweet-preprocessor, to clean the tweets. This package removes emoticons, user mentions, and smileys from tweets. We kept the hashtags as they can be a good indicator of conspiracy tweets. Next, we replaced the numbers by a special token "NUM" to reduce the number of unique tokens.

TF-IDF Vectorization

Using TF-IDF, we converted the tweets into word vectors. We removed stop words and also applied the stemmer before vectorization. To reduce the feature count, we confined the n-gram range to 2. The resulting vectorization contains 2,081 features.

Question 1: Linear Regressions

As RQ2 deals with a classification problem, linear regressions are not applicable to solve this problem.

Question 2: Logistic Regression and NB

Logistic Regression

To fit the word vectors into a logistic regression model, we split them into training and testing portions following 70%-30% rule. So, the training set contains 830 samples (519 non-conspiracy, 311 conspiracy) and the testing set has 357 samples (228 non-conspiracy, 129 conspiracy). We have used the "sklearn" python package for applying the logistic regression algorithm. The intercept is -0.56. As we have a very large number of features, it's not feasible to show all the coefficients of each feature. So, we performed the significance test and based on the p-values ($p<0.05$) we found 190 features to be significantly important. The following table shows the top 10 significant features (descending order) with their coefficient scores.

Feature	Coefficient	Odd-ratio	p-value
issu	-0.30	0.74	0.00049
hide	1.12	3.05	0.0012
week	-0.62	0.54	0.0013
family china	0.33	1.40	0.0015
armed coronavirus	0.33	1.40	0.0015
kill famili	0.33	1.40	0.0015
knows big	0.33	1.40	0.0015
said differ	0.33	1.40	0.0015
tech usa	0.33	1.40	0.0015
thebondfreak	0.33	1.40	0.0015

Table 3: Coefficients, odd-ratio and p-value for top 10 statistically significant features

From the coefficients, we can also predict the log odds of the outcome for a one unit increase in the features. For example, the coefficient of the word feature “hide” says that for every unit change in “hide”, the log odds of conspiracy vs. non-conspiracy. Table 3 also shows the odd-ratio of the features which can be interpreted same ways as the log-odds.

Figure 1 shows the performance of the logistic regression model in terms of precision, recall and accuracy. The precision, recall and average are quite satisfactory. The precision of the Conspiracy class denotes out of all the predictions the model makes for the conspiracy category, 97% time it is correct. And the recall represents that the model can identify 88% of the available conspiracy samples correctly.

	precision	recall	f1-score	support
Non-Conspiracy	0.93	0.98	0.96	228
Conspiracy	0.97	0.87	0.91	129
accuracy			0.94	357
macro avg	0.95	0.93	0.93	357
weighted avg	0.94	0.94	0.94	357

Avg. Accuracy: 0.9411764705882353

Figure 1: Performance of the Linear Regression

Naive Bayes

We used the same training testing set for Naive Bayes algorithm. The following figure shows the performance of it. The result looks amazing for the both categories with an average accuracy of 93%. The recall of the “Conspiracy” category is 100%, so it can identify all the available conspiracy tweets successfully, but the precision is 84% which describes the model classifies some tweets as “Conspiracy” mistakenly.

	precision	recall	f1-score	support
Non-Conspiracy	1.00	0.89	0.94	228
Conspiracy	0.84	1.00	0.91	129
accuracy			0.93	357
macro avg	0.92	0.95	0.93	357
weighted avg	0.94	0.93	0.93	357

Avg. Accuracy: 0.9327731092436975

Figure 2: Performance of the Naive Bayes without Laplace estimator

The above performance is reported without applying laplace estimator in Multinomial Naive Bayes algorithm. But after applying the estimator, the performance degrades a little bit.

	precision	recall	f1-score	support
Non-Conspiracy	0.83	0.99	0.90	228
Conspiracy	0.96	0.64	0.77	129
accuracy			0.86	357
macro avg	0.90	0.81	0.83	357
weighted avg	0.88	0.86	0.85	357

Avg. Accuracy: 0.8599439775910365

Figure 3: Performance of the Naive Bayes with Laplace estimator

Question 3. Decision Trees and Random Forests

Decision Trees

We have shuffled the dataset using the python package “pandas”. Then we splitted the dataset into training and testing using 70%-30% proportions. The distribution of the shuffled dataset is similar to the previous dataset. Figure 4 shows the distribution of a feature word “accident” to prove the claim.

count	1187.000000	count	1187.000000
mean	0.001977	mean	0.001977
std	0.017400	std	0.017400
min	0.000000	min	0.000000
25%	0.000000	25%	0.000000
50%	0.000000	50%	0.000000
75%	0.000000	75%	0.000000
max	0.253816	max	0.253816

Figure 4: Distribution of the feature word ‘accident’ before (left) and after shuffle(right)

We trained the decision tree algorithm using the training set. The size of the tree is 459. Figure 5 shows a glimpse of the generated tree. The tree can be interpreted as the combination of if-else rules. For example, the following tree can be interpreted like:

```
If W__infect <= 0.05:
        If W__ve <= 0.07:
                If W__wuhan virus <= 0.16
```

To show the better structure of the tree, we generated a flow-chart of the tree structure (Figure 6).

```
|--- W__infect <= 0.05
|   |--- W__ve <= 0.07
|   |   |--- W__wuhan virus <= 0.16
|   |   |   |--- W__china coronavirus <= 0.16
|   |   |   |   |--- W__corona <= 0.31
|   |   |   |   |   |--- W__prepar <= 0.17
|   |   |   |   |   |   |--- W__citi <= 0.33
|   |   |   |   |   |   |   |--- W__numnum <= 0.32
|   |   |   |   |   |   |   |   |--- W__numnumnumnumncov <= 0.05
|   |   |   |   |   |   |   |   |   |--- W__happen <= 0.30
|   |   |   |   |   |   |   |   |   |   |--- W__sad <= 0.40
|   |   |   |   |   |   |   |   |   |   |   |--- truncated branch of depth 90
|   |   |   |   |   |   |   |   |   |   |--- W__sad >  0.40
|   |   |   |   |   |   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |   |   |   |   |--- W__happen >  0.30
|   |   |   |   |   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |   |   |   |--- W__numnumnumnumncov >  0.05
```

Figure 5: Decision Tree Structure

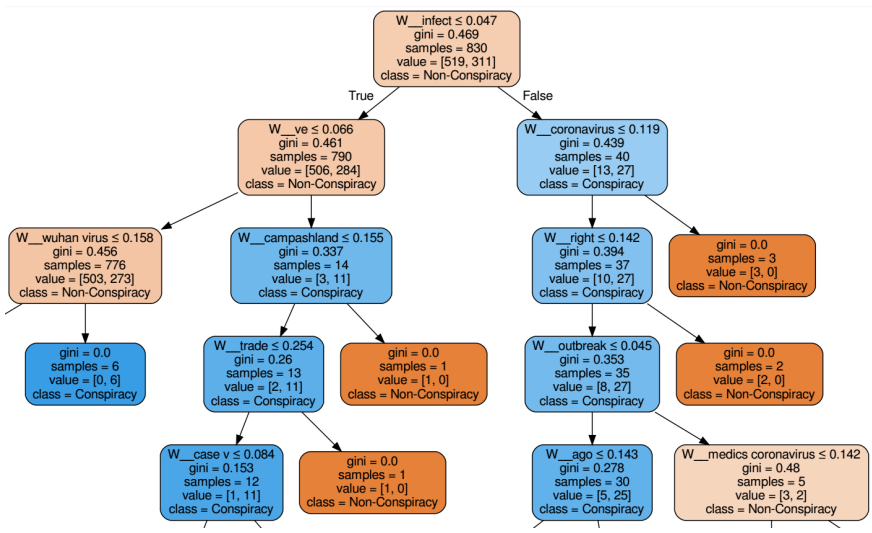


Figure 6: Decision Tree Visualization

The training of the decision tree incurs around 89% accuracy but it drops for testing (60%). Figure 7 shows the comparison of training and testing results. The training performance is way ahead than the testing.

	precision	recall	f1-score	support
Non-Conspiracy	0.88	0.95	0.91	519
Conspiracy	0.90	0.78	0.84	311
accuracy			0.89	830
macro avg	0.89	0.86	0.87	830
weighted avg	0.89	0.89	0.88	830

Avg. Accuracy: 0.8855421686746988

	precision	recall	f1-score	support
Non-Conspiracy	0.68	0.71	0.70	228
Conspiracy	0.44	0.40	0.42	129
accuracy			0.60	357
macro avg	0.56	0.56	0.56	357
weighted avg	0.59	0.60	0.59	357

Avg. Accuracy: 0.5994397759103641

Figure 7: Performance comparison of training (left) and testing (right) of Decision Tree

Boosting

Figure 8 shows the performance of the boosting for various numbers of trees. The performance of the boosting based on the accuracy is not consistent. For a higher number of trees (n = 150), the accuracy and precision increases but the recall drops when compared to a lower number of trees (n = 100).

	count	accuracy	precision	recall
0	100	0.809524	0.465517	0.868852
1	110	0.789916	0.465517	0.866667
2	120	0.806723	0.508621	0.828571
3	130	0.823529	0.500000	0.789474
4	140	0.803922	0.534483	0.867647
5	150	0.815126	0.534483	0.830986

Figure 8: Performance of Boosting

Bagging and Random Forest

Figure 9 shows the performance comparison of the Bagging and Figure 10 shows the comparison for Random Forest Classifier. Again there is no consistent increase or decrease of the performance with the increase of tree numbers for both Bagging and Random Forest. The best accuracy for bagging can be achieved when tree count is 120, but the precision and recall are achieved when the tree count are 140 and 110 respectively. For random forest, best accuracy and precision are achieved when the count is 140 and the recall at 1.

	count	accuracy	precision	recall
0	100	0.840336	0.646552	0.812500
1	110	0.865546	0.672414	0.898876
2	120	0.854342	0.715517	0.843750
3	130	0.868347	0.663793	0.831579
4	140	0.840336	0.681034	0.826087
5	150	0.840336	0.637931	0.852273

	count	accuracy	precision	recall
0	100	0.789916	0.474138	0.803030
1	110	0.789916	0.448276	0.880597
2	120	0.803922	0.474138	0.809524
3	130	0.798319	0.431034	0.789474
4	140	0.823529	0.491379	0.777778
5	150	0.789916	0.448276	0.820896

Table 4 shows the top 5 important features of random forest classifiers for different tree counts.

Tree #100	Tree #110	Tree #120	Tree #130	Tree #140	Tree #150
coronavirus	coronavirus	coronavirus	coronavirus	coronavirus	coronavirus
china coronavirus	infect	china	china	infect	china
china	china coronavirus	peopl	china coronavirus	china coronavirus	infect
think	china	chines	think	peopl	china coronavirus
peopl	happen	news	news	china	news

Table 4: Important Features for different random forest

Performance Comparison

Table 5 shows the performance comparison of the different models we explored. The performance of the decision tree is the worst among all the classifiers. But no classifier achieved the best performance alone for all the metrics we used. Logistic regression outperforms other models in terms of precision and accuracy. But naive bayes achieves the best recall. Note that we here mentioned the best performing models of each classifier only and for precision and recall we used the macro-average. Bagging and boosting perform almost similar but bagging shows lower performance than bagging in terms of recall. Overall, Logistic regression is the best performer. Naive bayes is the second best. But Naive bayes are smooth and faster for text processing. So, I have decided to go with the Naive Bayes.

Model	Precision	Recall	Accuracy
Logistic Regression	**0.95**	0.93	**0.94**

Naive Bayes	0.92	**0.95**	0.93
Decision Tree	0.56	0.56	0.60
Boosting	0.82	0.53	0.83
Bagging	0.85	0.72	0.84
Random Forest	0.82	0.49	0.78

Table 5: Performance comparison of the models

TWEETS- FEAR CAUSING OR FEAR RELIEVING

Our research question 3 asks if tweets can be categorized as Fear Causing or Fear Relieving.
# of Fear Causing Tweets: 251
# of Fear Relieving Tweets: 250

LINEAR REGRESSIONS
This did not apply due to classification data

Logistic Regressions
# of Training Data = 350
# of Testing Data = 151
Intercept = -0.05
Following figure shows the statistically significant features with their p-value, coefficients and odd-ratio. Coefficients are known as log-odd and expresses the intuition like odd-ratio. Odd-ratio is the exponent value of the coefficients and represents the outcome for a unit increase for a feature. For example, the odd-ratio of the feature 'coronavirus' denotes the odds of fear-causing and fear-relieving for a unit change in 'coronavirus'.

	p-value	Coefficier	Odd-ratio
coronavirus	7.5	0.36	1.43
negat	0.0005	0.97	2.64
like	0.0023	0.99	2.69
infect	0.003	0.83	2.85
human	0.004	0.23	1.26
just	0.005	1.01	2.77
unit	0.005	0.83	0.43
say	0.009	0.75	2.1
beij	0.01	0.68	1.9
shi p	0.01	0.77	2.17

Figure: Coefficients, odd-ratio, and p-value of the top features

The following figure shows the performance of the logistic regression model. Precision, recall and accuracy are all 0.60. Causing class has more precision than relieving class, but the scenario is opposite for recall.

	precision	recall	f1-score	support
Relieving	0.59	0.63	0.61	75
Causing	0.61	0.58	0.59	76
accuracy			0.60	151
macro avg	0.60	0.60	0.60	151
weighted avg	0.60	0.60	0.60	151

Avg. Accuracy: 0.6026490066225165

Figure: Performance of the logistic Regression

NAIVE BAYES

Multinomial Naive Bayes without laplace estimator

	precision	recall	f1-score	support
Relieving	0.64	0.69	0.67	75
Causing	0.67	0.62	0.64	76
accuracy			0.66	151
macro avg	0.66	0.66	0.66	151
weighted avg	0.66	0.66	0.66	151

Avg. Accuracy: 0.6556291390728477

Multinomial Naive Bayes with laplace estimator

	precision	recall	f1-score	support
Relieving	0.61	0.63	0.62	75
Causing	0.62	0.61	0.61	76
accuracy			0.62	151
macro avg	0.62	0.62	0.62	151
weighted avg	0.62	0.62	0.62	151

Avg. Accuracy: 0.6158940397350994

So, adding the laplace estimator degrades the performance.

DECISION TREES

We have shuffled the dataset using the python package “pandas”. Then we splitted the dataset into training and testing using 70%-30% proportions. The distribution of the shuffled dataset is similar to the previous dataset. Figure 4 shows the distribution of a feature word “accident” to prove the claim.

```
count    501.000000
mean       0.014005
std        0.078896
min        0.000000
25%        0.000000
50%        0.000000
75%        0.000000
max        0.619008
Name: W__world, dtype: float64
```

We trained the decision tree algorithm using the training set. The size of the tree is 459. Figure 5 shows a glimpse of the generated tree. The tree can be interpreted as the combination of if-else rules. For example, the following tree can be interpreted like:

```
|--- W__numnum <= 0.36
|   |--- W__measur <= 0.17
|   |   |--- W__peopl <= 0.43
|   |   |   |--- W__wuhan coronavirus <= 0.44
|   |   |   |   |--- W__offici <= 0.44
|   |   |   |   |   |--- W__author <= 0.20
|   |   |   |   |   |   |--- W__time <= 0.32
|   |   |   |   |   |   |   |--- W__report <= 0.35
|   |   |   |   |   |   |   |   |--- W__lie <= 0.22
|   |   |   |   |   |   |   |   |   |--- W__novel coronavirus <= 0.40
|   |   |   |   |   |   |   |   |   |   |--- W__spread <= 0.45
|   |   |   |   |   |   |   |   |   |   |   |--- truncated branch of depth 72
|   |   |   |   |   |   |   |   |   |   |--- W__spread >  0.45
|   |   |   |   |   |   |   |   |   |   |   |--- class: 0
|   |   |   |   |   |   |   |   |   |--- W__novel coronavirus >  0.40
|   |   |   |   |   |   |   |   |   |   |--- class: 0
|   |   |   |   |   |   |   |   |--- W__lie >  0.22
|   |   |   |   |   |   |   |   |   |--- class: 0
|   |   |   |   |   |   |   |--- W__report >  0.35
|   |   |   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |   |--- W__time >  0.32
|   |   |   |   |   |   |   |--- W__time <= 0.77
|   |   |   |   |   |   |   |   |--- class: 0
|   |   |   |   |   |   |   |--- W__time >  0.77
|   |   |   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- W__author >  0.20
|   |   |   |   |   |   |--- class: 0
|   |   |   |   |--- W__offici >  0.44
|   |   |   |   |   |--- class: 0
|   |   |   |--- W__wuhan coronavirus >  0.44
|   |   |   |   |--- class: 0
|   |   |--- W__peopl >  0.43
|   |   |   |--- class: 1
|   |--- W__measur >  0.17
|   |   |--- class: 1
|--- W__numnum >  0.36
|   |--- class: 0
```

Training Confusion Matrix

	precision	recall	f1-score	support
Relieving	0.98	0.99	0.98	175
Causing	0.99	0.98	0.98	175
accuracy			0.98	350
macro avg	0.98	0.98	0.98	350
weighted avg	0.98	0.98	0.98	350

Avg. Accuracy: 0.9828571428571429

Testing Confusion Matrix

	precision	recall	f1-score	support

```
   Relieving       0.51      0.49      0.50        75
     Causing       0.51      0.53      0.52        76

    accuracy                           0.51       151
   macro avg       0.51      0.51      0.51       151
weighted avg       0.51      0.51      0.51       151

Avg. Accuracy:  0.5099337748344371
```

BOOSTING

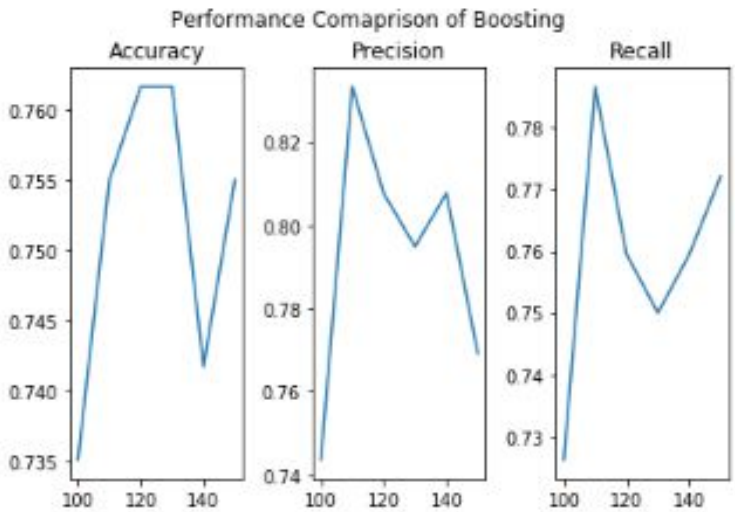
This table shows the performance of the boosting for various numbers of trees. The performance of the boosting based on the accuracy is not consistent. For a higher number of trees (n = 150), the accuracy and precision increases but the recall drops when compared to a lower number of trees (n = 100).

	count	accuracy	precision	recall
0	100	0.649007	0.512821	0.750000
1	110	0.682119	0.564103	0.753846
2	120	0.715232	0.641026	0.741935
3	130	0.675497	0.653846	0.750000
4	140	0.695364	0.628205	0.746032
5	150	0.682119	0.641026	0.734375

BAGGING AND RANDOM FOREST

The following shows the performance comparison of the Bagging and Figure 10 shows the comparison for Random Forest Classifier. Again there is no consistent increase or decrease of the performance with the increase of tree numbers for both Bagging and Random Forest. The best accuracy for bagging can be achieved when tree count is 120, but the precision and recall are achieved when the tree count are 140 and 110 respectively. For random forest, best accuracy and precision are achieved when the count is 140 and the recall at 1.

	count	accuracy	precision	recall
0	100	0.735099	0.743590	0.726190
1	110	0.754967	0.833333	0.786667
2	120	0.761589	0.807692	0.759494
3	130	0.761589	0.794872	0.750000
4	140	0.741722	0.807692	0.759494
5	150	0.754967	0.769231	0.772152



Tree Count 100	Tree Count 110	Tree Count 120	Tree Count 130	Tree Count 140	Tree Count 150
W__coronavirus	W__coronavirus	W__coronavirus	__coronavirus	W__coronavirus	W__coronavirus
W__china	W__china	W__china	W__china	W__china	W__china
W__peopl	W__peopl	W__peopl	W__peopl	W__peopl	W__virus
W__numnum	W__numnum	W__virus	W__virus	W__virus	W__peopl
W__virus	W__virus	W__num	W__numnum	W__numnum	W__numnum
W__chines	W__wuhan	W__chines	W__chines	W__chines	W__chines
W__num	W__num	W__numnum	W__num	W__num	W__wuhan
W__case	W__chines	W__spread	W__wuhan	W__wuhan	W__num
W__wuhan	W__case	W__time	W__case	W__time	W__time
W__infect	W__time	W__wuhan	W__outbreak	W__spread	W__case

PERFORMANCE COMPARISON

MODEL	PRECISION	RECALL	ACCURACY
Logistic Regression	0.60	0.60	0.60
Naïve Bayes	0.66	0.66	0.66
Decision Tree	0.51	0.51	0.51
Boosting	0.68	0.64	0.73
Bagging	**0.75**	0.77	0.77
Random Forest	0.74	**0.79**	**0.78**

RQ4: Classifying Information Seeking Tweets

This research question deals with whether a tweet is Information Seeking or not. It is useful for the concerned authorities to know if people seek information on the pandemic COVID-19 and to address their concern. Hence, with the help of this question, we are going to predict if a tweet is information seeking or not.

We randomly sampled 499 tweets from our dataset and labeled them as Information Seeking or NO (0 = Not Seeking, 1 = Seeking)

We were able to label:
455 - Non Seeking
43 - Seeking

Tweet Preprocessing

We used a python package, tweet-preprocessor, to clean the tweets. This package removes emoticons, user mentions, and smileys from tweets. We kept the hashtags as they can be a good indicator of conspiracy tweets. Next, we replaced the numbers by a special token "NUM" to reduce the number of unique tokens.

TF-IDF Vectorization

Using TF-IDF, we converted the tweets into word vectors. We removed stop words and also applied the stemmer before vectorization. To reduce the feature count, we confined the n-gram range to 2. The resulting vectorization contains 2,081 features.

Question 1: Linear Regression

As RQ4 deals with a classification problem, linear and multivariate regressions are not applicable to solve this problem.

Question 2: Logistic Regression and NB

Logistic Regression

We have used the "sklearn" python package for applying the logistic regression algorithm. The intercept is -0.68. As we have a very large number of features, it's not feasible to show all the coefficients of each feature. So, we performed the significance test and based on the p-values ($p<0.05$) we found 190 features to be significantly important. The following table shows the top 5 significant features (descending order) with their coefficient scores.

Feature	**Co-efficient**	**Odd-Ratio**	**p-value**
truth	1.0882564546687705	2.9690928090864888	1.4626341237622353e-65
account follow	49074552381357134	1.6335336043067608	1.3652753762354188e-54
crossstreamw	0.49074552381357134	1.6335336043067608	1.3652753762380442e-54
crossstreamw suggest	0.49074552381357134	1.6335336043067608	1.365275376238297e-54
suggest english	0.49074552381357134	1.6335336043067608	1.3652753762390744e-54

The following confusion matrix depicts the performance of the logistic regression. The precision, recall and average are doing quite well. The precision of the Non-Information Seeking class denotes out of all the predictions the model makes for the conspiracy category, 94% time it is correct. And the recall represents that the model can identify 85% of the available information seeking samples correctly.

```
                        precision    recall  f1-score   support

Non-Information Seeking       0.94      1.00      0.97       141
    Information Seeking       1.00      0.85      0.92        60

               accuracy                           0.96       201
              macro avg       0.97      0.93      0.94       201
           weighted avg       0.96      0.96      0.95       201

Avg. Accuracy:  0.9552238805970149
```

Naive Bayes

We used the same training testing set for Naive Bayes algorithm. The following confusion matrix can depict the performance:

Fig 1: Without laplace estimator:

```
                        precision    recall  f1-score   support

Non-Information Seeking       1.00      0.96      0.98       141
    Information Seeking       0.92      1.00      0.96        60

               accuracy                           0.98       201
              macro avg       0.96      0.98      0.97       201
           weighted avg       0.98      0.98      0.98       201

Avg. Accuracy:  0.9751243781094527
```

Fig 2- With laplace estimator:

```
                        precision    recall  f1-score   support

Non-Information Seeking       0.92      0.99      0.96       141
    Information Seeking       0.98      0.80      0.88        60

               accuracy                           0.94       201
              macro avg       0.95      0.90      0.92       201
           weighted avg       0.94      0.94      0.93       201

Avg. Accuracy:  0.9353233830845771
```

We can see that after applying the Laplace estimator, the performance worsens a little bit. Overall, without the estimator, the accuracy of the model is 98%.

Decision Tree

We have shuffled the dataset using the python package “pandas”. Then we splitted the dataset into training and testing using 70%-30% proportions. The distribution of the shuffled dataset is similar to the previous dataset. Figure 4 shows the distribution of a feature word “trutht” to prove the claim.

count	670.000000	count	670.000000
mean	0.015414	mean	0.015414
std	0.109517	std	0.109517
min	0.000000	min	0.000000
25%	0.000000	25%	0.000000
50%	0.000000	50%	0.000000
75%	0.000000	75%	0.000000
max	1.000000	max	1.000000

The tree can be interpreted as the combination of if-else rules.The tree can be interpreted a follows:

```
|--- W__rip <= 0.34
|   |--- W__confirm <= 0.38
|   |   |--- W__hope <= 0.45
|   |   |   |--- W__fear <= 0.53
|   |   |   |   |--- W__tell <= 0.56
|   |   |   |   |   |--- W__pandem <= 0.20
|   |   |   |   |   |   |--- W__government ag <= 0.24
|   |   |   |   |   |   |   |--- W__coronavirus <= 0.23
|   |   |   |   |   |   |   |   |--- W__new <= 0.27
|   |   |   |   |   |   |   |   |   |--- W__china <= 0.37
|   |   |   |   |   |   |   |   |   |   |--- W__number <= 0.47
|   |   |   |   |   |   |   |   |   |   |   |--- truncated branch of depth 81
|   |   |   |   |   |   |   |   |   |   |--- W__number >  0.47
|   |   |   |   |   |   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |   |   |   |   |--- W__china >  0.37
|   |   |   |   |   |   |   |   |   |   |--- W__watch <= 0.27
|   |   |   |   |   |   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |   |   |   |   |   |--- W__watch >  0.27
|   |   |   |   |   |   |   |   |   |   |   |--- class: 0.0
|   |   |   |   |   |   |   |   |--- W__new >  0.27
|   |   |   |   |   |   |   |   |   |--- W__coronavirus <= 0.06
|   |   |   |   |   |   |   |   |   |   |--- W__just <= 0.26
```

We have the performance comparison of training (left) and testing (right) of Decision Tree:

	precision	recall	f1-score	support
Non-Information Seeking	0.87	0.96	0.91	314
Information Seeking	0.91	0.70	0.79	155
accuracy			0.88	469
macro avg	0.89	0.83	0.85	469
weighted avg	0.88	0.88	0.87	469

Avg. Accuracy: 0.8763326226012793

	precision	recall	f1-score	support
Non-Information Seeking	0.69	0.74	0.71	141
Information Seeking	0.27	0.23	0.25	60
accuracy			0.59	201
macro avg	0.48	0.49	0.48	201
weighted avg	0.57	0.59	0.58	201

Avg. Accuracy: 0.5870646766169154

The training of the decision tree incurs around 88% accuracy but it drops for testing (59%).

Boosting

The figure below shows the performance of the boosting for various numbers of trees. The performance of the boosting based on the accuracy is not consistent. For a higher number of trees (n = 150), the accuracy and precision increases and the recall drops but suddenly increases when n = 150.

	count	accuracy	precision	recall
0	100	0.855721	0.607843	0.820513
1	110	0.855721	0.607843	0.815789
2	120	0.880597	0.647059	0.800000
3	130	0.865672	0.627451	0.815789
4	140	0.860697	0.627451	0.711111
5	150	0.875622	0.607843	0.837838

Figure: Performance of Boosting

Bagging and Random Forest

The following two figures show the performance comparison of the Bagging and the comparison for Random Forest Classifier respectively. Again there is no consistent increase or decrease of the performance with the increase of tree numbers for both Bagging and Random Forest. The best accuracy for bagging can be achieved when tree count is 150, but the precision and recall are achieved when the tree count is 130. For random forest, best accuracy, precision and recall are achieved when the count is 130.

	count	accuracy	precision	recall
0	100	0.835821	0.470588	0.880000
1	110	0.840796	0.470588	0.870968
2	120	0.845771	0.509804	0.823529
3	130	0.840796	0.549020	0.875000
4	140	0.860697	0.529412	0.823529
5	150	0.865672	0.529412	0.848485

Figure: Performance of Bagging

	count	accuracy	precision	recall
0	100	0.830846	0.568627	0.783784
1	110	0.840796	0.607843	0.794118
2	120	0.870647	0.549020	0.891892
3	130	0.850746	0.607843	0.810811
4	140	0.850746	0.568627	0.794872
5	150	0.840796	0.568627	0.783784

Figure: Performance of Random Forest

Comparison of the models:

The following table shows the performance comparison of all of our models. The performance of the decision tree is the worst among all the classifiers. Multinomial Naive Bayes outperforms other models in terms of recall and accuracy. But logistic regression achieves the best precision. Note that we here mentioned the best performing models of each classifier only and for precision and recall we used the macro-average. Bagging and boosting perform almost similar but bagging shows lower performance than bagging in terms of recall. Overall, Naive Bayes is the best performer and Logistic regression is the second best.

Model	**Precision**	**Recall**	**Accuracy**
Logistic Regression	**0.97**	0.93	0.96
Naive Bayes	0.96	**0.98**	**0.98**
Decision Tree	0.48	0.49	0.59
Boosting	0.87	0.54	0.84
Bagging	0.60	0.83	0.87
Random Forest	0.60	0.81	0.85